```
public enum Genre {
  ADVENTURE,
  FANTASY,
  SCIENCE_FICTION
}
```

Finally, implement the business logic to retrieve book quotes in a new `QuoteService` class. Quotes will be defined and stored in a static in-memory list.

Listing 16.3 The business logic to query book quotes

```
@Service
public class QuoteService {
  private static final Random random = new Random();
  private static final List<Quote> quotes = List.of(      ❶
    new Quote("Content A", "Abigail", Genre.ADVENTURE),
    new Quote("Content B", "Beatrix", Genre.ADVENTURE),
    new Quote("Content C", "Casper", Genre.FANTASY),
    new Quote("Content D", "Dobby", Genre.FANTASY),
    new Quote("Content E", "Eileen", Genre.SCIENCE_FICTION),
    new Quote("Content F", "Flora", Genre.SCIENCE_FICTION)
  );

  public Flux<Quote> getAllQuotes() {
    return Flux.fromIterable(quotes);                     ❷
  }

  public Mono<Quote> getRandomQuote() {
    return Mono.just(quotes.get(random.nextInt(quotes.size() - 1)));
  }

  public Mono<Quote> getRandomQuoteByGenre(Genre genre) {
    var quotesForGenre = quotes.stream()
      .filter(q -> q.genre().equals(genre))
      .toList();
    return Mono.just(quotesForGenre.get(
     random.nextInt(quotesForGenre.size() - 1)));
  }
}
```

❶ Stores a list of quotes in memory

❷ Returns all quotes as a reactive data stream

NOTE Since the focus of this example is the native-image compilation with GraalVM and Spring Native, we'll keep it simple and skip the persistence layer. Feel free to expand it on your own. For